MODULE 13 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Step Functions and Orchestration

Standard against Express, three integration patterns, and error handling that lives outside your code.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 13 of 34

admin@sthithapragna.com

Step Functions moves **coordination** out of your code. Retries, branching, waiting and error handling become configuration you can see.

Amazon States Language

- A state machine is a **JSON document** in Amazon States Language, or ASL.
- It names a **StartAt** state and a map of states, each naming its **Next** or setting **End**.
- Each state receives JSON input and produces JSON output, which becomes the next state's input.
- Workflow Studio builds this visually, and the console renders any definition as a diagram.
- Payloads between states are capped at **256 KB**. Large data goes to S3 with a pointer passed.

> ON THE EXAM
>
> That 256 KB ceiling shapes real designs. Passing an S3 key rather than a document is the standard workaround, and it appears in scenarios.

The eight state types

STATE	WHAT IT DOES
Task	Does the work. Calls a Lambda function or another AWS service
Choice	Branches on a condition. The if statement
Parallel	Runs several branches at once and waits for all of them
Map	Runs the same steps once per item in an array
Wait	Pauses for a duration, or until a timestamp
Pass	Passes input through, optionally injecting fixed data. Useful for testing
Succeed	Ends the execution successfully
Fail	Ends the execution with an error name and cause

ON THE EXAM

Parallel runs different branches concurrently; Map runs the same branch over many items. Confusing those two is a common exam mistake.

Standard or Express workflow?

STANDARD

Long, auditable, exactly-once

- Runs up to one year
- Exactly-once execution
- Full visual execution history
- Priced per state transition
- Supports the callback pattern

EXPRESS

Short, high-volume, cheap

- Runs up to 5 minutes
- At-least-once execution
- History only in CloudWatch Logs
- Priced by duration and memory
- No callback pattern

THE TIEBREAKER

Duration and volume decide it. Over 5 minutes or needing exactly-once means Standard. High-volume short-lived event processing means Express, at a fraction of the cost.

Express workflows in detail

- **Synchronous Express** returns the result to the caller. Good behind API Gateway.
- **Asynchronous Express** returns immediately and the result goes to CloudWatch Logs.
- **At-least-once** means a state can run twice, so tasks must be idempotent.
- No execution history in the console. You query CloudWatch Logs instead.
- Typical use: high-volume event ingestion, IoT data, streaming transformations.

> ON THE EXAM
>
> The idempotency requirement is the real cost of Express. Standard gives exactly-once precisely so you do not have to think about it.

Service integrations

- **Optimised integrations** for common services, with extra behaviour built in.
- **AWS SDK integrations** reach over 14,000 API actions across more than 220 services.
- So a Task state can write to DynamoDB or publish to SNS with no function at all.
- Fewer Lambda functions means less code, lower cost and fewer cold starts.
- The state machine's **IAM role** needs permission for whatever it calls.

> ON THE EXAM
>
> When a scenario describes a Lambda function that only makes one AWS API call inside a workflow, a direct service integration removes it entirely.

The three integration patterns

PATTERN	BEHAVIOUR
`Request Response`	Call and continue immediately. The default
`.sync`	Call and wait for the job to finish. Run a Job
`.waitForTaskToken`	Call, then pause until something calls SendTaskSuccess

ON THE EXAM

The suffix in the resource ARN selects the pattern. Reading that suffix tells you whether the workflow waits, and for what.

The callback pattern

- Step Functions passes a **task token** to the target and then pauses the execution.
- It stays paused, at no compute cost, until **SendTaskSuccess** or **SendTaskFailure** arrives.
- A Standard workflow can wait like this for up to **one year**.
- The classic use is **human approval**: send an email, wait for someone to click.
- **Standard workflows only.** Express does not support callbacks.

> ON THE EXAM
>
> Any scenario with a manual approval step, or an external system that will answer later, is describing this pattern. Nothing else waits indefinitely for free.

Retry

- A **Retry** block lists error names, and Step Functions retries the state on those errors.
- **IntervalSeconds**, **MaxAttempts** and **BackoffRate** control the schedule.
- **States.ALL** matches everything. **States.TaskFailed** and **States.Timeout** are more specific.
- Retriers are evaluated in order, so put specific error names before States.ALL.
- **JitterStrategy** spreads retries out, which prevents a thundering herd.

> ON THE EXAM
>
> This replaces retry loops inside your function code. A Lambda function that retries internally is doing work the state machine should own.

Catch

- A **Catch** block routes to a different state when an error is not resolved by retries.
- Different error names can route to different handlers: notify, compensate, or fail cleanly.
- **ResultPath** on a Catch decides whether the handler sees the error, the input, or both.
- Retry runs first. Catch applies only once retries are exhausted.
- Without a Catch, an unhandled error terminates the whole execution.

> ON THE EXAM
>
> Retry then Catch is the order, and it mirrors ordinary code: try the operation a few times, then handle the failure. Both are configuration, not code.

Map and Parallel

- **Map** runs the same sub-workflow once per item in an array.
- **MaxConcurrency** limits how many run at once. Set it to protect downstream systems.
- **Parallel** runs several **different** branches simultaneously and waits for all.
- In Parallel, if any branch fails the whole state fails, unless that branch catches it.
- Parallel output is an array, one element per branch, in branch order.

> ON THE EXAM
>
> Map is for a list of similar work. Parallel is for a fixed set of different work. If the number of items varies at runtime, it is Map.

Distributed Map

- A Map state in **Distributed mode**, running child workflow executions rather than inline iterations.
- Reads its items directly from **Amazon S3**: a JSON array, a CSV file, or a list of objects.
- Scales to **10,000 parallel child executions**, far past inline Map.
- **ItemBatcher** groups items so one child handles many, cutting cost.
- A **tolerated failure threshold** lets the job continue past a percentage of failures.

> ON THE EXAM
>
> Inline Map is bounded by the 256 KB payload, so the array must fit in state. Distributed Map reads from S3 and has no such ceiling.

Moving data between states

- **InputPath** selects part of the input before the state sees it.
- **Parameters** builds the payload actually sent to the integration.
- **ResultSelector** reshapes what the integration returned.
- **ResultPath** decides where that result is placed relative to the original input.
- **OutputPath** selects what is finally passed on to the next state.

> ON THE EXAM
>
> ResultPath is the one that matters most. Setting it to $ replaces the input entirely; setting it to $.result keeps the input and nests the result inside it.

Currency note: JSONata and Variables

- **The examinable model:** JSONPath, with the five fields on the previous slide.
- **Variables** let one state assign a value that any later state can read directly.
- That removes the need to thread data through every intermediate state.
- **JSONata** is an alternative query language, set with the **QueryLanguage** field.
- Variables total **10 MB per execution**, which is far beyond the 256 KB state payload.

> ON THE EXAM
>
> Genuinely useful in practice, because the 256 KB payload limit was a real constraint. QueryLanguage defaults to JSONPath, so the exam's model is still the default.

Intrinsic functions

- Built-in functions usable inside Parameters, avoiding a Lambda function for trivial work.
- **States.Format** builds a string from a template and values.
- **States.Array**, **States.ArrayLength** and **States.ArrayPartition** manipulate arrays.
- **States.JsonToString** and **States.StringToJson** convert between the two.
- **States.UUID** and **States.MathRandom** generate values inside the workflow.

> ON THE EXAM
>
> The pattern to recognise: a Lambda function that only formats a string or counts an array can be replaced by an intrinsic function.

The limits worth knowing

LIMIT	VALUE
Standard execution duration	1 year
Express execution duration	5 minutes
Payload between states	256 KB
Distributed Map child executions	10,000 in parallel
Execution history entries	25,000 for Standard, then the execution fails
Standard pricing	Per state transition
Express pricing	Per request, plus duration and memory

ON THE EXAM

The 25,000 history entry limit catches long-running loops. A workflow that polls in a Wait loop for weeks will eventually hit it.

Step Functions or just code?

USE STEP FUNCTIONS WHEN

Coordination is the problem

- Several steps with branching
- Retries differ per step
- Something must wait, or wait for a human
- You need to see where it failed
- Steps span multiple services

KEEP IT IN CODE WHEN

The logic is one unit

- A single logical operation
- Sub-second latency matters
- Very high volume, cost-sensitive
- The steps never fail independently
- State transition cost would dominate

THE TIEBREAKER

Step Functions charges per state transition on Standard, so a chatty workflow of trivial steps can cost more than the work it coordinates.

Currency note: Durable Functions, revisited

- **The examinable rule:** multi-step or long-running workflows use Step Functions.
- **Lambda Durable Functions**, announced December 2025, keep the workflow in ordinary code.
- They checkpoint automatically and can wait up to a year without billing idle time.
- **Not in exam guide v2.1.** Answer Step Functions.
- Step Functions still wins on visual history, service integrations without code, and Distributed Map.

> ON THE EXAM
>
> Module 6 flagged this from the Lambda side. Repeated here because this is the module where the wrong instinct would actually cost you a mark.

Two of these are decided by **how long something waits**. Find the duration before you look at the options.

> An expense approval workflow validates a claim, then requires a manager to approve it before payment. Managers are emailed a link and typically respond within a day, but occasionally take up to three days.
>
> The team wants no compute running while waiting, wants to see which claims are stuck awaiting approval, and wants the workflow to continue automatically once a manager clicks.

How should the approval step be implemented?

A. A Wait state of 72 hours, then a Task state that checks an approvals table

B. An Express workflow that polls an approvals table every 30 seconds

C. A Lambda function that loops, sleeping and re-checking until approval arrives

D. A Task state using .waitForTaskToken, with the token sent in the approval email

◆ PAUSE AND DECIDE

How should the approval step be implemented?

A Wait 72 hours then check

Every claim waits the full three days even when approved in ten minutes. It also fails if a manager takes slightly longer.

B Express workflow polling

Express workflows stop at five minutes, so this cannot span three days. Polling also burns invocations to discover nothing has changed.

C Lambda sleep loop

Lambda stops at 15 minutes, and you pay for every second of sleeping. Paying a function to wait is the anti-pattern this pattern removes.

D .waitForTaskToken

The execution pauses with no compute running, appears in the console as awaiting callback, and resumes the moment SendTaskSuccess arrives.

THE SIGNAL

Three requirements: no compute while waiting, visible status, and immediate resumption. The callback pattern is the only option that delivers all three.

A telemetry pipeline receives about 8,000 events per second. Each event runs through the same four steps: validate, enrich from DynamoDB, transform, and write to Kinesis Data Firehose. The whole thing takes under two seconds per event.

Occasional duplicate processing is acceptable because the downstream store is idempotent. Cost at this volume is the primary concern.

Which workflow type should the team use?

A. A Standard workflow, one execution per event

B. An asynchronous Express workflow, one execution per event

C. A Standard workflow with a Distributed Map over batched events

D. A single Lambda function performing all four steps in sequence

◆ PAUSE AND DECIDE

Which workflow type should the team use?

A. Standard per event

Standard is priced per state transition. Four transitions x 8,000 per second is an enormous bill for work that finishes in two seconds.

B. Express per event

Under five minutes, at-least-once is acceptable, and duration-based pricing is far cheaper at this volume. Exactly what Express was built for.

C. Distributed Map over batches

Distributed Map suits a large finite dataset, not a continuous stream. It adds orchestration overhead to a steady flow.

D. One Lambda function

The reasonable-looking answer, and it puts branching and retry logic back inside code. Defensible, but it abandons the visibility the workflow provides.

THE SIGNAL

Every clause points at Express: under 5 minutes, duplicates acceptable, cost primary. The scenario grants at-least-once explicitly, which is the permission slip.

> A media company must re-encode about 5 million video files listed in an S3 bucket. Each file takes two to ten minutes in an ECS task. The job should run as fast as the encoding fleet allows, and a small percentage of corrupt files should not stop the run.
>
> The team needs a report of which files failed. The list of objects is far too large to pass as workflow input.

How should the job be orchestrated?

A. A Distributed Map state reading the object list from S3, with a tolerated failure threshold

B. An inline Map state over an array of all 5 million object keys

C. A Parallel state with one branch per file

D. An Express workflow triggered once per object by S3 event notifications

◆ PAUSE AND DECIDE

How should the job be orchestrated?

A Distributed Map from S3

Reads items directly from S3 so nothing passes through state, scales to 10,000 child executions, and the tolerated failure threshold handles corrupt files.

B Inline Map

The array would have to fit in the 256 KB state payload. Five million keys is orders of magnitude past that.

C Parallel with one branch per file

Parallel branches are defined statically in the definition. You cannot write five million of them, and any failure fails the state.

D Express per object

Encoding takes up to ten minutes, which exceeds the five-minute Express limit. It also gives no aggregate view of the run.

THE SIGNAL

'Far too large to pass as workflow input' names the 256 KB limit without saying it, and only Distributed Map reads its items from S3 instead.

> A payment workflow calls a third-party gateway. The gateway occasionally returns a 503 that succeeds on retry, and occasionally returns a permanent CardDeclined error. Currently a Lambda function implements its own retry loop, and declined cards surface as generic failures.
>
> The team wants transient errors retried with increasing delays, and declined cards routed to a separate notification step, without adding code to the function.

How should this be configured?

A Increase the Lambda function timeout and retry attempts in the function code

B Wrap the Task in a Parallel state with two branches, one for each error type

C Add a Retry block for the transient error with BackoffRate, and a Catch block routing CardDeclined to the notification state

D Configure a Lambda destination on failure pointing at the notification function

◆ PAUSE AND DECIDE

How should this be configured?

A Retry inside the function

Keeps coordination logic in code, which the scenario explicitly wants removed. A retrying function also holds compute open while it waits.

B Parallel with two branches

Parallel runs branches concurrently, it does not select between them. It cannot express 'if this error, go here'.

C Retry plus Catch

Retry with BackoffRate handles the transient 503 with increasing delays; Catch routes CardDeclined to its own state. Both are configuration, not code.

D Lambda destination on failure

Destinations apply to asynchronous invocation. Step Functions invokes Lambda synchronously, so the destination never fires.

THE SIGNAL

Retry then Catch is the pairing: retry the transient, route the permanent. Option D is the module 5 fact reappearing - destinations need async invocation.

What to carry forward

- **Over 5 minutes or exactly-once means Standard.** High volume and short means Express.
- **.waitForTaskToken** pauses for up to a year at no compute cost. Human approval lives here.
- **Retry then Catch.** Transient errors retried, permanent errors routed.
- Inline Map is bounded by the 256 KB payload. Distributed Map reads items from S3.
- Standard is priced per state transition, so chatty workflows get expensive.

> ON THE EXAM
>
> Next: containers, Elastic Beanstalk and the rest of compute - the last Domain 1 module, and another selection-heavy one.